IT2080 - IT Project

Assignment 2- Progress Review
Group 76

Presentation Date: 22/09/2025
Project Title: **The PurePortion** Smart Food Waste Reduction System

IT Number	Name	Email	Contact Number
IT23860032	N.S.Ambagahawaththa	it23860032@my.sliit.lk	0761224358
IT23858848	K.K.A.H.Janeesha	it23858848@my.sliit.lk	0740754727
IT23857230	Damsara T.M.D.	it23857230@my.sliit.lk	0713534939
IT23617100	Pathiranage S.S.	it23617100@my.sliit.lk	0717892110
IT23858398	Dilranga M.M.P.M	it23858398@my.sliit.lk	0742931829

1.Introduction

What is Pure portion ?

Food waste has become a serious problem in many parts of the world, including Srilanka. Every day large amount of food is throw away by households, restaurants, hotels, and catering services.one of the main reasons for this is that people often cook more food than needed. This happens because they do not know the exact amount of food needs per person. As a results extra food is leftover and usually goes to waste.
In Sri Lanka, this issue is common in homes as well as in food businesses. Most families prepare
meals without proper planning. they do not keep track of the ingredients they have and they do not reuse leftover food properly. restaurants and catering services also face the same problem. They buy and use large amounts of food without accurate portion control, which leads to high costs and food waste.

To solve this issue we decided to build a web based system called The PurePortion.
This system is designed to help different types of users such as households and restaurants plan their meals in a smart way . It help to user calculate the correct amount of food they needed for the number of people they cooking for. It will also helps manage the kitchens inventory and track expired items and reduce food waste by suggest recipes from leftovers and even manage financial records related to food.

The idea for this project came from analyzing daily problems faced by the people when cooking. Many users want a simple and smart tool that can guide them in cooking the right amount of food and saving money and reducing waste. Our goal is to build an all in one solution that can be used by any household or food service provider to manage their kitchen efficiently and responsibly.
In this project 5 group members will work together to develop different parts of the system. Each member will take responsibility for one major function with full CRUD (Create /Read/ Update/Delete) operations. Those features include user management, portion calculation, recipe management and inventory tracking and leftover suggestions and financial management.
By creating The PurePortion web application we aim to promote better food habits and save money also reduce kitchen waste and support environmental sustainability. It is a practical solution that can help both families and businesses in their daily food planning.

2.Progress

1) User Story

1) User Management
    a) As a household user, I want to register and log in so that I can have a personal dashboard for my meal planning.
    b) As a restaurant owner, I want to add my staff as users so that they can manage inventory and cooking tasks and Manage Finances.

2) Inventory Management
    a) As a **user** I want to **add** and **edit or delete ingredients** so that **I can keep my inventory updated** and get Inventory list as PDF
    b) As a **user** I want to **get expiry reminder** so that **I do not waste food by forgetting items in storage**.

3) Portion Calculation
    a) As a **cook** I want to **calculate the right portion for the number of people** so that **I do not overcook and waste food** and I want to know the exact cost for per person
    b) I want to download pdf report for each meal Plan

4) Recipe Management
    a) As a **home cook** I want to **see recipes based on what is in fridge** so that **I can avoid wasting leftovers**
    b) As a **chef** I want to **get recipe with estimated costs** so that **I can manage expenses**.

5) Leftover Management
    a) As a **restaurant owner** I want to **donate excess food** so that **it can be used by those in need instead of being wasted**
    b) As an **NGO or charity** I want to **request food donations** so that **we can provide food to people in need**.
    c) As a **household user or restaurant user** I want to **get recipe suggestions from an AI chat bot** so that **I can reuse my leftovers in creative ways**

6) Finance Management
    a) As a **user** I want to **log grocery expenses** so that **I can track weekly and monthly food costs.**
    b) As a User I want to Pay for my Staff Salaries with EPF ETF Bonuses, Allowances
    c) As a **restaurant manager**, I want to **see cost per meal reports** so that **I can control overhead expenses.**

Progress Table:

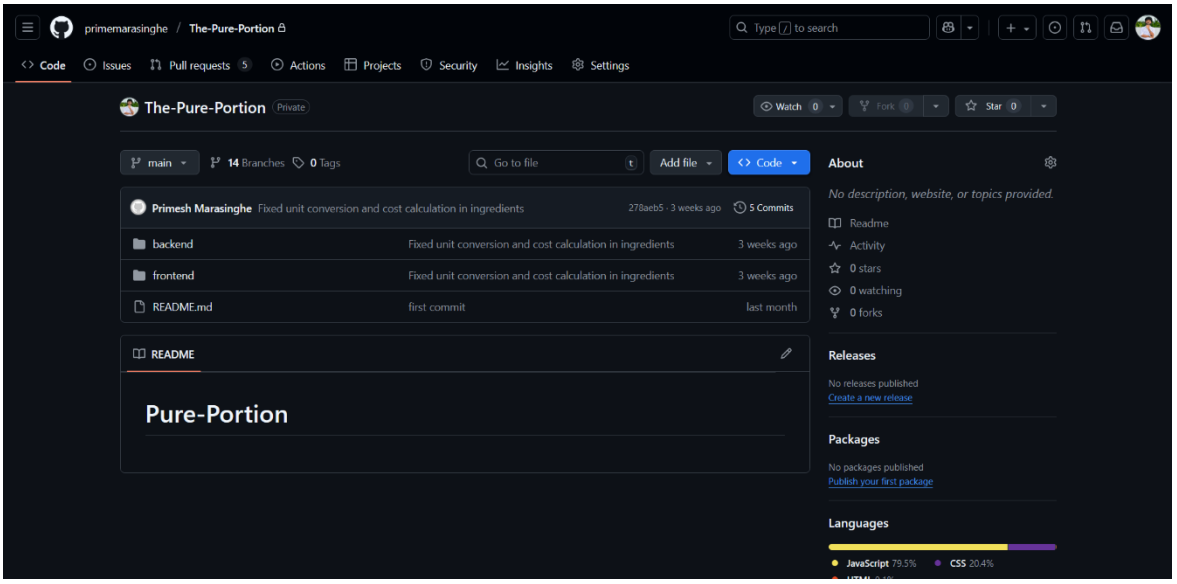
High level user story / function	Completed (Y/N +comment)	Responsible member
User Management		
As a household user I want to register and log in so that I can have a personal dashboard for my meal planning.	Y	Pathiranage S.S. (IT23617100)
As a restaurant owner I want to add my staff as users so that they can manage inventory, cooking tasks and finances.	Y	
Optimize PDF report generation for user data.	N (Pending)	
Enhance Admin Panel with better controls.	N (Pending)	
Upgrade UI/UX for intuitive navigation.	N (Pending)	
Refined version with stability, performance, usability improvements.	N (Pending)	
Inventory Management		
As a user, I want to add, edit or delete ingredients so that I can keep my inventory updated.	Y	K.K.A.H.Janeesha (IT23858848)
As a user I want to get expiry reminders so that I do not waste food.	Y	
Export inventory list as PDF.	Y	
Duplicate item detection.	N (Pending)	
Upgrade UI/UX design for better responsiveness.	N (Pending)	
Portion Calculation		
As a cook, I want to calculate portions for people so that I do not overcook and waste food.	Y	Dilranga M.M.P.M (IT23858398)
I want to know the exact cost per person.	Y	

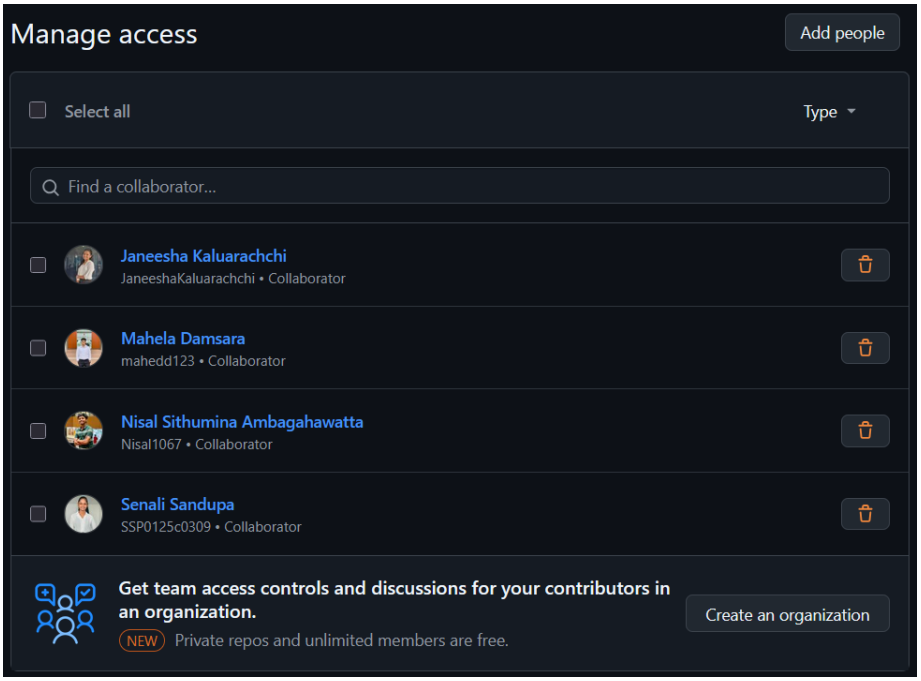
Download PDF report for each meal plan.	Y	
Save portion plans for future reuse.	N (Pending)	
Display total cost at bottom based on selected ingredients.	N (Pending)	
Recipe Management		
As a home cook, I want to see recipes based on my fridge inventory so I avoid wasting leftovers.	Y	Damsara T.M.D (IT23857230)
As a restaurant chef, I want recipe ideas with estimated costs to manage expenses better.	Y	
Image upload for recipes.	Y	
Real time inventory sync with recipes.	Y	
Export recipe reports as PDF.	Y	
Send and receive notification from Inventory.	N (Pending)	
Upgrade UI/UX for recipe features.	N (Pending)	
Leftover Management		
As a restaurant owner, I want to donate excess food so that it's not wasted.	Y	N.S.Ambagahawaththa (IT23860032)
As an NGO or charity, I want to request food donations to feed people.	Y	
As a household/restaurant user, I want recipe suggestions from AI chatbot for leftovers.	Y	
Add and manage leftover donations.	Y	
Export "My Donations" and "My Requests" as PDF.	Y	
Implement Request Donation feature.	N (Pending)	
Add real time notifications for approvals.	N (Pending)	

Upgrade UI/UX for leftover section.	N (Pending)	
Finance Management		
As a user, I want to log grocery expenses to track food costs.	Y	Dilranga M.M.P.M (IT23858398)
As a user, I want to pay staff salaries with EPF, ETF, bonuses and allowances.	Y	
As a restaurant manager, I want cost per meal reports to control expenses.	Y	
Display today profit, monthly profit, expenses, income.	Y	
Show staff count and salary budget.	Y	
Manage staff payments with full breakdown.	Y	
Generate finance PDF reports.	Y	
View recent finance records and delete entries.	Y	
Download individual staff salary reports.	N (Pending)	
Auto update finance summary when portions are created.	N (Pending)	

2. Repository

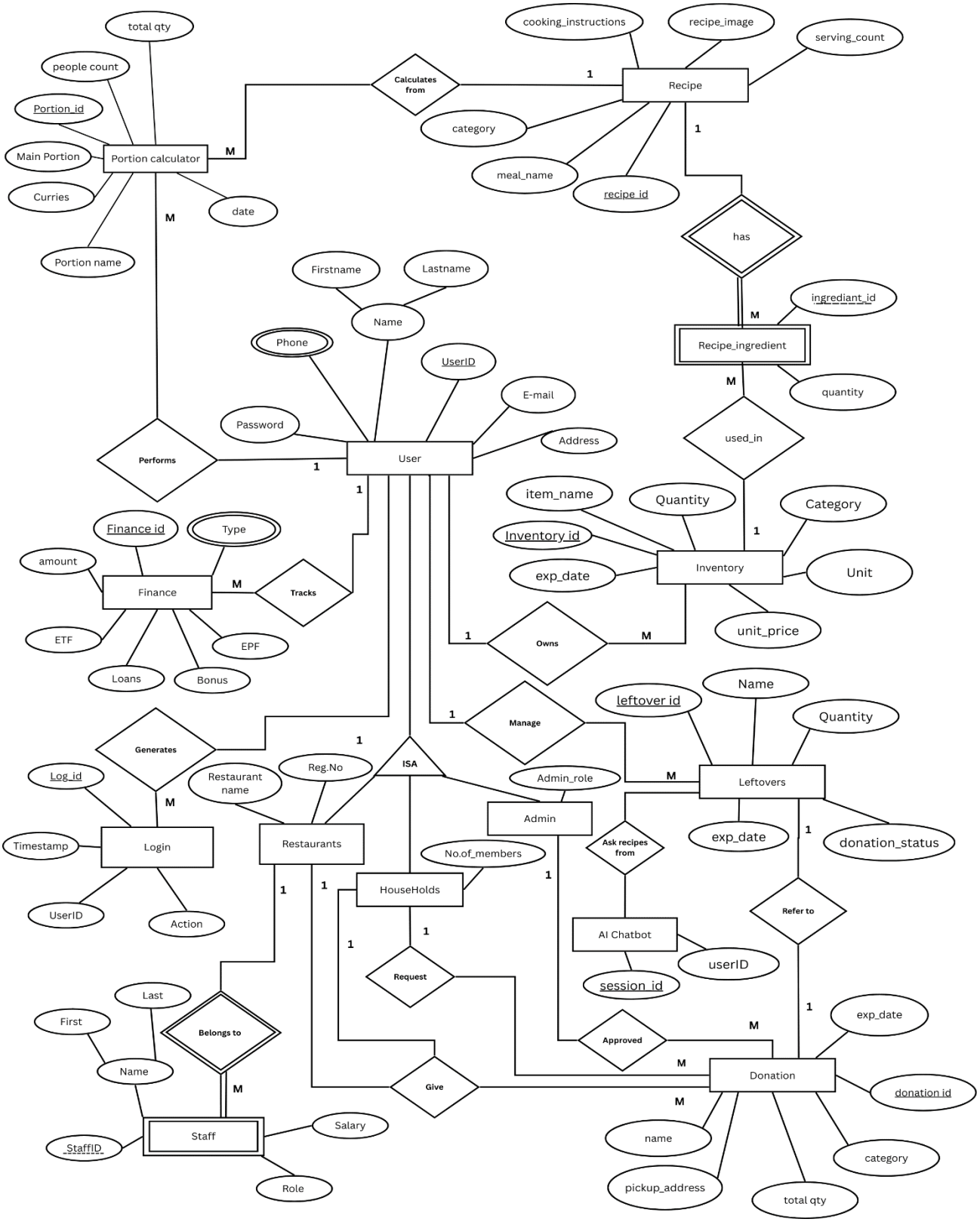
Link - https://github.com/primemarasinghe/The-Pure-Portion



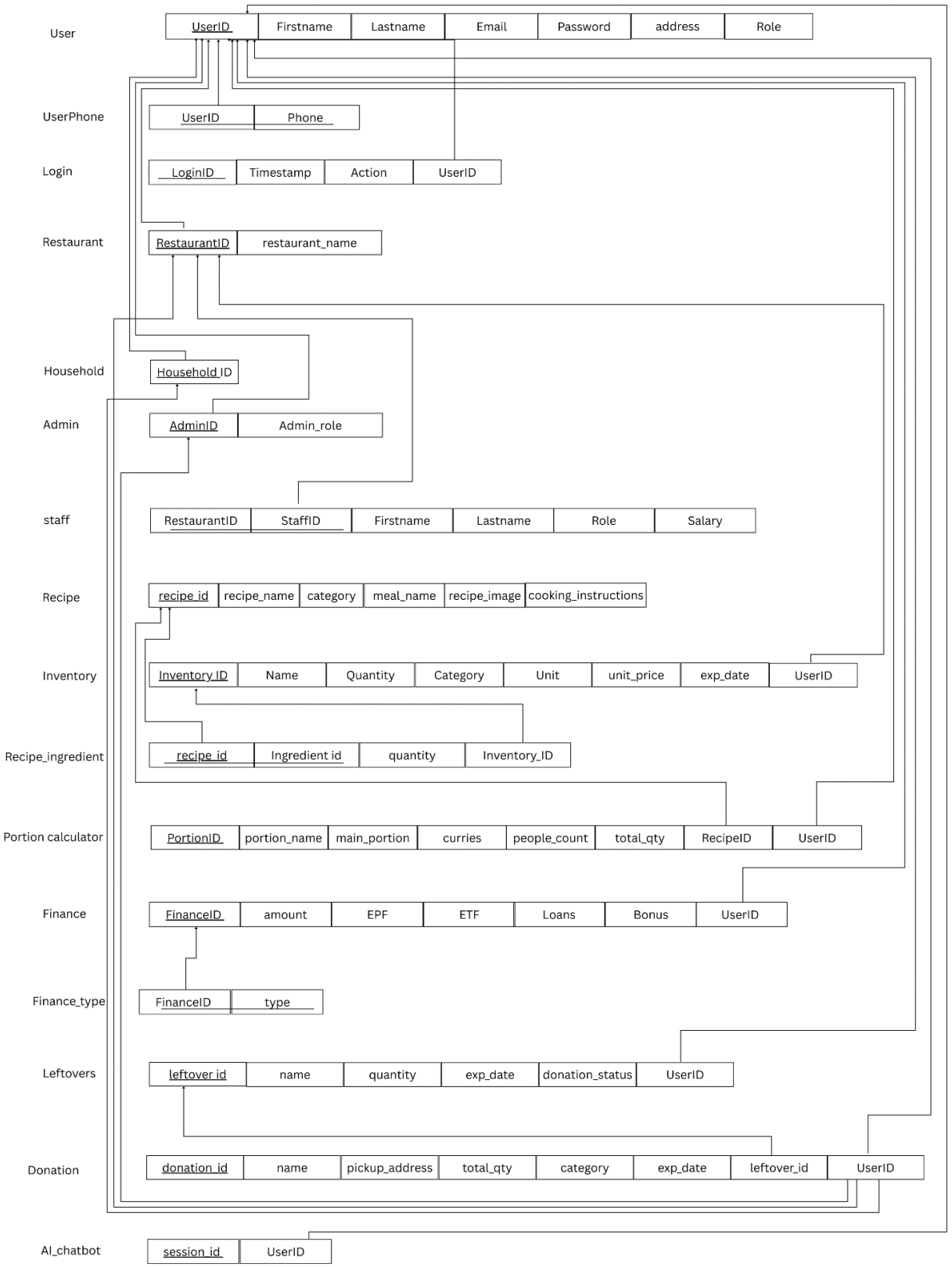


3. System Design & Technical Artifacts

1. ER Diagram



2. Normalized Schema



3. Test Case Design

User Management:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-101	Household user registration	First Name: Nimal Last Name: Perera Email:nimal@mail.com Phone: 0771234567 Password:******	1. Open household Registration form 2. Fill all valid fields 3. Click "Submit"	Account created and redirected to dashboard	201 Created	Yes
TC-102	Household user registration with invalid phone	Phone: 07712345	1. Open Registration Form 2. Enter invalid Phone 3. Submit	Error: "Phone number must be 10 digits"	400 Bad Request	Yes
TC-103	Household user registration with invalid name	First Name: Nimal@12	1. Fill first/last name with special chars 2. Submit	Error: "Invalid characters in name"	400 Bad Request	Yes
TC-104	Password mismatch validation	Password: 12345 Confirm Password: 1234	1. Enter passwords differently 2. Submit form	Error: "Passwords do not match"	400 Bad Request	Yes
TC-105	Login with valid credentials	Email: nimal@mail.com Password: 12345	1. Open login page 2. Enter valid credentials 3. Click "Login"	Successful login redirects to dashboard	200 OK	Yes
TC-106	Login with invalid credentials	Email: nimal@mail.com Password: wrongpass	1. Enter email and wrong password 2. Submit	Error: "Incorrect Password"	401 Unauthorized	Yes
TC-107	Restaurant registration valid	Restaurant Name: FoodHub Owner Name: Saman, Phone: 0764586987 Email:nis@gmail.com	1. Open restaurant registration form 2. Fill all valid fields 3. Submit	Restaurant account created successfully	201 Created	Yes
TC-108	Add staff to restaurant	Staff Name, Email, Phone, Position	1. Login as Restaurant Owner 2. Open Add Staff form 3. Fill details 4. Submit	Staff added successfully and email notification sent	201 Created	Yes

Inventory Management:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-201	Add inventory item valid	Item Name: Rice, Category: Grains, Qty: 10, Expiry: future	1. Open inventory page 2. Click Add Item 3. Fill valid details 4. Submit	Item saved successfully	201 Created	Yes
TC-202	Add inventory item with past expiry	Expiry: 2024-01-01	1. Open Add Item form 2. Enter past expiry 3. Submit	Error: "Expiry date cannot be in the past"	400 Bad Request	Yes
TC-203	Item name validation	Item Name: Rice123$	1. Enter invalid item name 2. Submit	Error: "Invalid characters in item name"	400 Bad Request	Yes
TC-204	Export inventory as PDF	Inventory exists	1. Click "Export PDF"	Inventory exported as PDF	200 OK	Yes
TC-205	Low stock alert	Qty: 2, Min: 5	1. Add inventory 2. Check alerts	"Low stock" notification displayed	200 OK	Yes
TC-206	Quantity limit validation	Qty: 2000	1. Enter Qty >1000 2. Submit	Error: "Max quantity is 1000"	400 Bad Request	Yes
TC-207	Supplier name validation	Supplier: ABC123	1. Enter invalid supplier name 2. Submit	Error: "Letters only allowed"	400 Bad Request	Yes
TC-208	Purchase date validation	Purchase Date: past	1. Enter purchase date 2. Submit	Error: "Purchase date cannot be past"	400 Bad Request	Yes

Portion Calculator:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-301	Calculate portion valid	Plan Name: Lunch People: 5	1. Open Portion Calculator 2. Fill plan name and number of people 3. Submit	Portion calculated, inventory updated	200 OK	Yes
TC-302	People limit validation	People: 25000	1. Enter people>20000 2. Submit	Error: "Maximum 20000 allowed"	400 Bad Request	Yes
TC-303	Generate meal plan report	Portion created	1. Click "Export PDF"	PDF downloaded with meal plan	200 OK	Yes
TC-304	Empty plan name validation	Plan Name: blank	1. Leave plan name blank 2. Submit	Error: "Plan name required"	400 Bad Request	Yes
TC-305	Plan name character validation	Plan Name: Lunch@123	1. Enter invalid characters 2. Submit	Error: "Invalid characters in plan name"	400 Bad Request	Yes
TC-306	Auto update inventory	Inventory exists	1. Create portion 2. Check inventory	Ingredients deducted correctly	200 OK	Yes
TC-307	Notification after portion creation	Portion created	1. Create portion 2. Check notifications	Low stock or portion-created notification received	200 OK	Yes
TC-308	Low stock alert	Inventory near min	1. Create portion using near min stock 2. Check alerts	Low stock alert displayed	200 OK	Yes

Leftover Management:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-501	Donate food valid	Name: Rice Pack, Qty: 5kg, City: Colombo	1. Open Donate Food form 2. Fill valid details 3. Submit	Donation saved successfully	201 Created	Yes
TC-502	City selection validation	City: Imaduwa	1. Enter invalid city 2. Submit	Error: "City not allowed"	400 Bad Request	Yes
TC-503	Request donation valid	NGO details + proof	1. Fill Request Donation form 2. Submit	Request saved successfully	201 Created	Yes
TC-504	AI recipe suggestion	Leftovers exist	1. Open AI chatbot 2. Enter leftovers	Suggested recipes displayed	200 OK	Yes
TC-505	Expiry date validation	Expiry past	1. Enter past expiry 2. Submit	Error: "Expiry cannot be in past"	400 Bad Request	Yes
TC-506	Food name validation	Name: Rice@123	1. Enter invalid name 2. Submit	Error: "Invalid characters"	400 Bad Request	Yes
TC-507	Proof document required	No document	1. Leave proof empty 2. Submit	Error: "Proof required"	400 Bad Request	Yes
TC-508	Urgency level validation	Blank	1. Leave urgency blank 2. Submit	Error: "Urgency required"	400 Bad Request	Yes

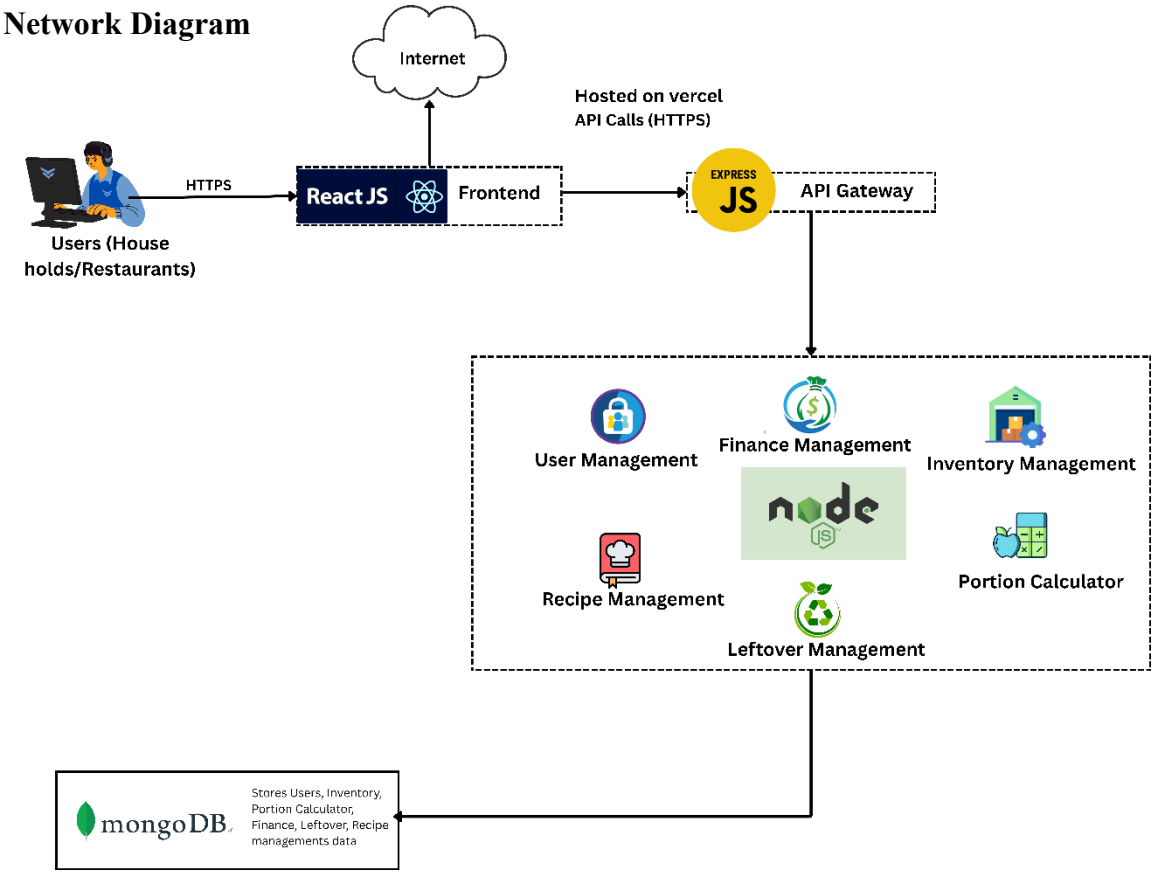
Finance Management:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-601	Add finance record	Type: Expense, Amount: 500	1. Open Finance page 2. Fill Add Record form 3. Submit	Record saved successfully	201 Created	Yes
TC-602	Staff salary with allowances	Basic: 50000, Allowance: 5000	1. Open Process Staff Payment 2. Enter details 3. Submit	Salary calculated correctly including EPF/ETF	200 OK	Yes
TC-603	Delete finance record	Existing record	1. Select record 2. Click Delete 3. Confirm	Record deleted successfully	200 OK	Yes
TC-604	Export finance report	Records exist	1. Click Export PDF	PDF downloaded	200 OK	Yes
TC-605	Amount validation	Amount: -1000	1. Enter negative amount 2. Submit	Error: "Amount must be positive"	400 Bad Request	Yes
TC-606	Category required	Blank	1. Leave category empty 2. Submit	Error: "Category required"	400 Bad Request	Yes
TC-607	Bonus calc invalid input	Amount = 0	1. Select staff 2. Enter 0 bonus 3. Submit	Error: "Amount must be >0"	400 Bad Request	Yes

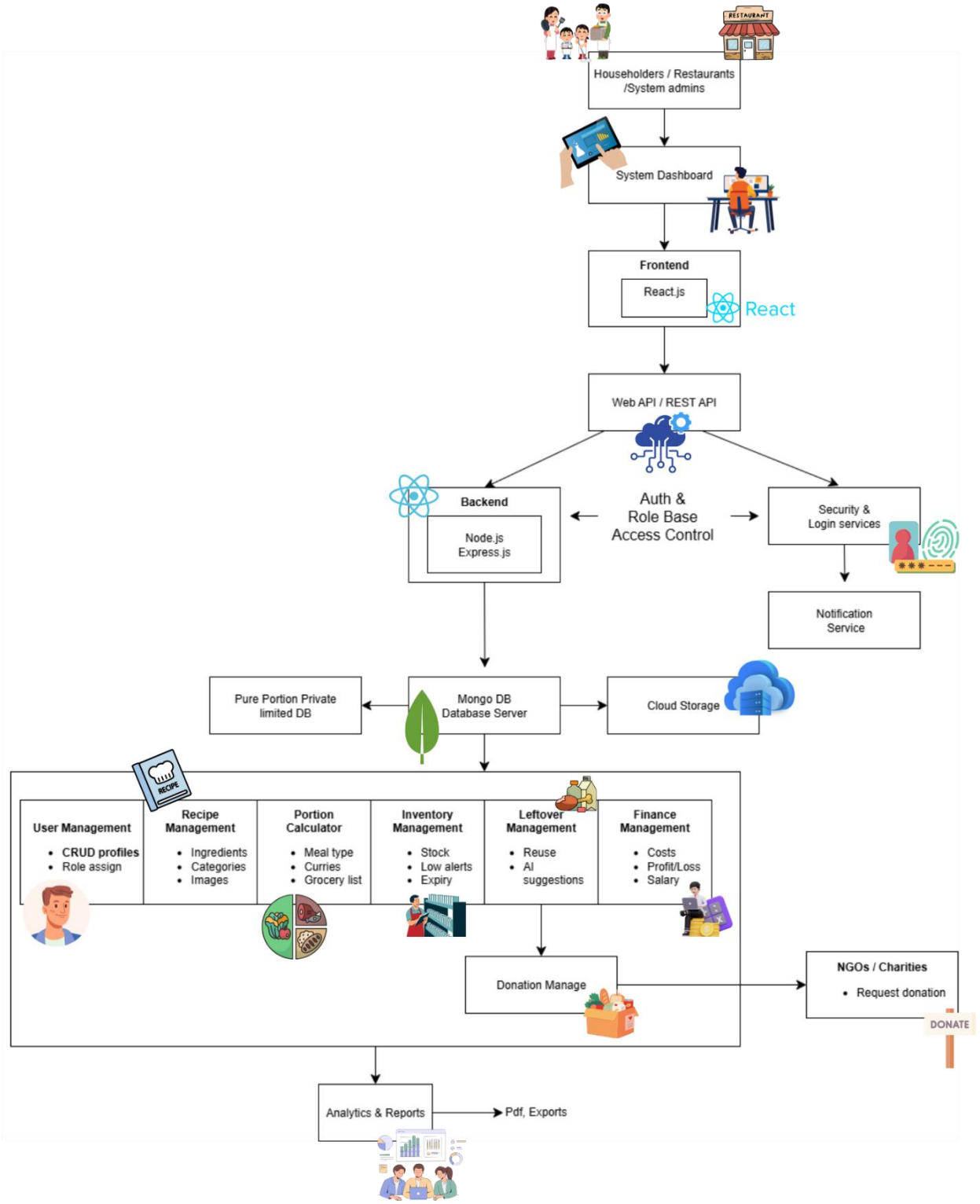
Recipe Management:

TC-ID	Description	Input Data	Test Steps	Expected Result	Error/Code	Automated
TC-401	Add recipe valid	Name: Chicken Curry, Ingredients selected	1. Open Recipe form 2. Fill details 3. Submit	Recipe saved successfully	201 Created	Yes
TC-402	Image upload validation	File: curry.jpg	1. Upload valid image 2. Submit	Recipe saved with image	200 OK	Yes
TC-403	Recipe cost calculation	Ingredients: Chicken 500g, Onion 100g	1. Select recipe ingredients 2. Check cost	Cost calculated correctly	200 OK	Yes
TC-404	Export recipe report	Recipe exists	1. Click Export PDF	Recipe PDF downloaded	200 OK	Yes
TC-405	Recipe name validation	Name: Curry@123	1. Enter invalid characters 2. Submit	Error: "Invalid characters"	400 Bad Request	Yes
TC-407	Serving size validation	Servings = 0	1. Enter servings = 0 2. Submit	Error: "Servings must be >0"	400 Bad Request	Yes
TC-408	Validate image format restriction	File: virus.exe	1. Upload unsupported image 2. Submit	Error: "Invalid file format"	400 Bad Request	Yes

4. Network Diagram

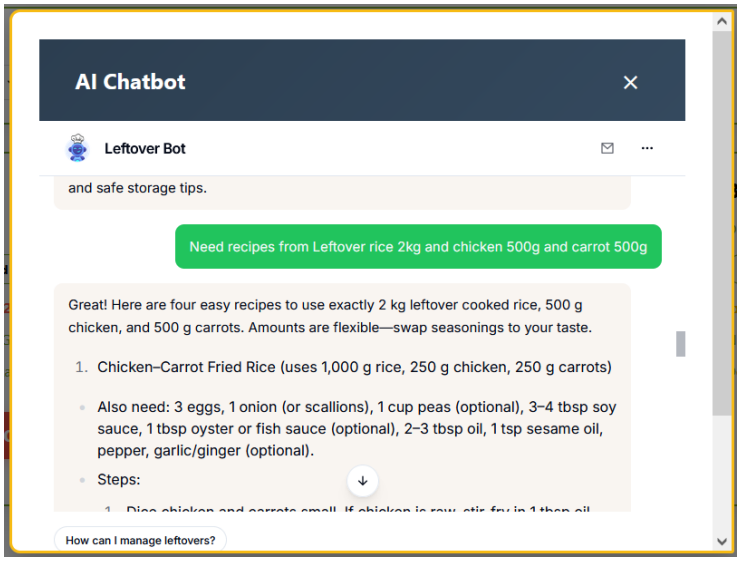


5. High Level System Design Diagram

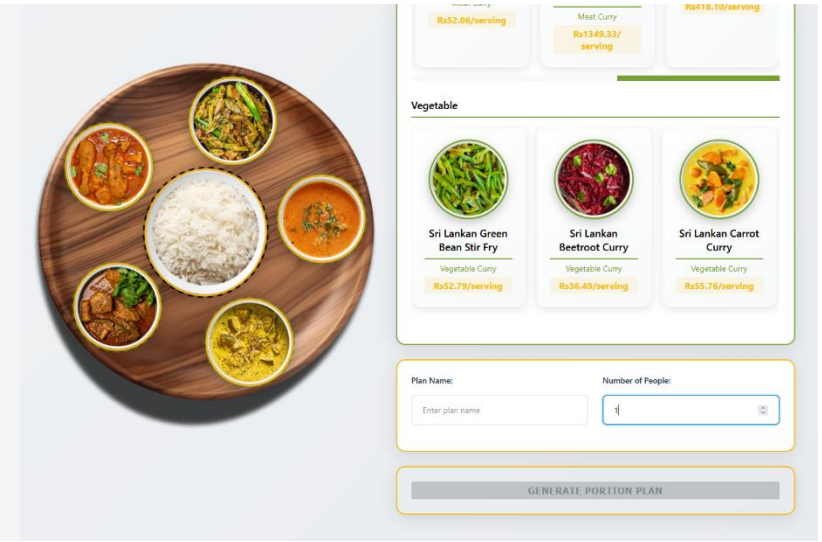


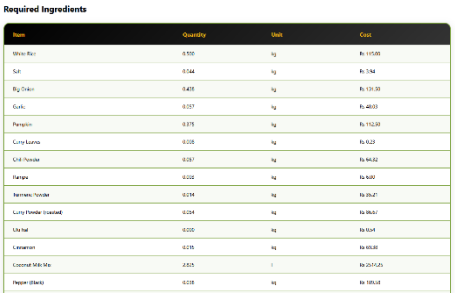
6. Innovative Features

a) **Leftover Based Recipe Suggestions** - The system leverages both standard recipes and an AI powered chatbot to suggest creative ways to reuse leftover ingredients. This directly reduces household and commercial food waste.



b) **Intelligent Portion Calculator** - A context aware portioning tool that adjusts quantities based on the number of people, meal type and ingredient availability. This prevents overcooking and minimizes unnecessary food preparation.





c) **Integrated Expense and Inventory Tracking** - A unified platform that automatically links ingredient usage with financial records, enabling users and businesses to monitor costs while managing stock efficiently.

d) **Sustainability Driven Design** - Every feature is built with environmental responsibility in mind, from minimizing food waste to promoting efficient resource usage, ultimately contributing to long term sustainability goals.

e) **Donation and Redistribution Network (Innovative Extension)** - Restaurants and households can donate excess food, while NGOs or individuals can request available donations. This extends the system impact beyond waste reduction, creating social value by supporting communities.

7. Commercialization Potential

Commercialization

Commercialization is the process of bringing innovative solutions into the market and making them accessible to real world users.
Pure Portion - Food Waste Management System is a web based platform designed to reduce food waste and promote sustainability across restaurants, Catering service and households. It provides real time food inventory Management, Portion Calculation, Recipe Management, Finance Management and Leftover Management.

Target Market

Restaurants, catering services and hotels aim to reduce food costs and meet sustainability goals.
Households that want smart monitoring of consumption and waste patterns.

Pain Points Addressed on our web application

Untracked food waste - Real time inventory monitoring and expiry alerts.
High disposal costs - Donation and redistribution features for surplus food.
Lack of awareness - Analytics dashboard showing waste patterns and cost impact.
Sustainability reporting challenges - Automated reports for compliance and CSR.

Unique Selling Proposition (USP)

Role based Access and Security - Restaurants, NGOs and households can all use the same application with role based access.
Customizable and Scalable - The platform can be adapted for restaurants, catering service and even households, without being limited to one type of food business.

8. Individual Member Contributions

User Management - (IT23617100 PATHIRANAGE S.S.)

The User Management function allows different types of users such as Restaurant, catering service, Householders and admin to register and log into the Pure Portion system. Households can register and they can select recipes and use the portion calculator.

Completion Level	In progress
User login and registration Based on the Role Type	Optimize PDF report generation to ensure proper formatting and consistent output.
User Profile Update	Enhance the Admin Panel with improved functionality and better user controls.
Staff Attendance and Restaurant Settings	Upgrade the overall UI/UX design to provide a more intuitive and user friendly experience.
Staff Management	Develop a refined version of the system with improved stability, performance and usability.
Admin Panel	

MongoDB Queries (Mongoose)

Update staff and Delete staff

```
const updateStaff = async (req, res) => {
  try {
    const restaurantId = new mongoose.Types.ObjectId(req.user.userId);
    const staffId = req.params.id;

    const staff = await Staff.findOne({ _id: staffId, restaurantId });
    if (!staff) return res.status(404).json({ message: 'Staff member not found' });
```

```
const deleteStaff = async (req, res) => {
  try {
    const restaurantId = new mongoose.Types.ObjectId(req.user.userId);
    const staffId = req.params.id;
```

Add Staff and Get All Staff

```
const addStaff = async (req, res) => {
  try {
    console.log('Add staff request received:', req.body);
    console.log('User from token:', req.user);
```

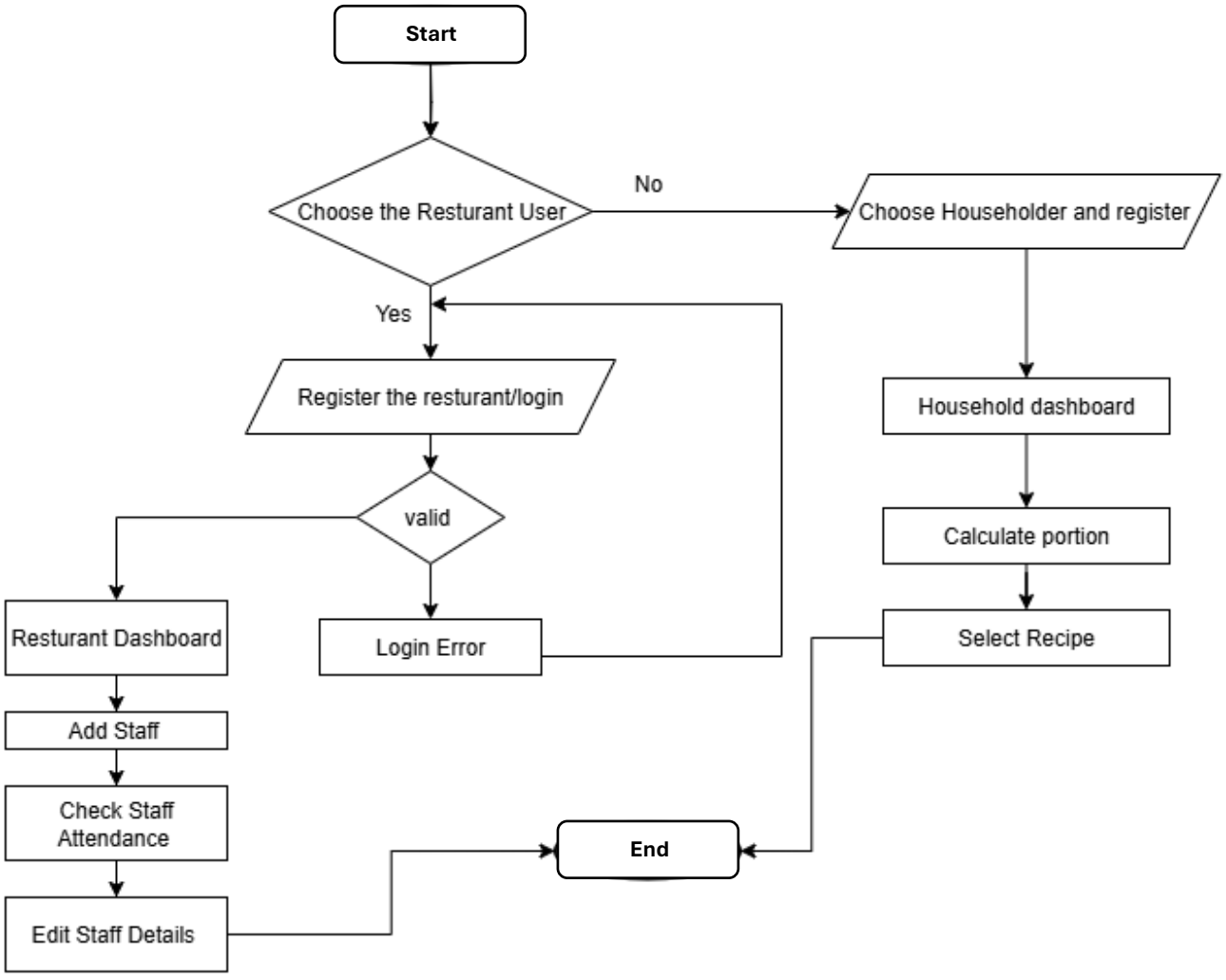
```
const getAllStaff = async (req, res) => {
```

Get Single Staff and Staff Stats

```
const getStaffById = async (req, res) => {
  try {
    const restaurantId = new mongoose.Types.ObjectId(req.user.userId);
    const staff = await Staff.findOne({ _id: req.params.id, restaurantId });
```

```
const getStaffStats = async (req, res) => {···
};

module.exports = {
  addStaff,
  getAllStaff,
  getStaffById,
  updateStaff,
  deleteStaff,
  getStaffStats,
  upload
};
```

Flow chart:



Inventory Management Module (IT23858848 - Janeesha K.K.A.H.)

Completion Level

The Inventory Management Module is currently **80% completed**. Most of the essential functionalities have been implemented and tested, while a few improvements and refinements are still pending before the final version.

Completed	To be Completed
Category based filtering has been developed, allowing items to be organized and searched more efficiently.	**Duplicate item detection** is still under development. This feature will ensure that the same item cannot be added multiple times, avoiding data redundancy.
The system can now **display expiring soon and low stock items**, helping users manage inventory in advance.	**UI/UX design of the inventory dashboard** needs to be improved to enhance navigation, responsiveness and overall user friendliness.
Export as PDF functionality has been implemented, making it easy to generate and download inventory reports.	
Auto update from portion calculation is fully functional, deducting stock automatically when portions are created.	
Notification alerts have been introduced to inform users about deducted stock and ingredients that are running low	

Queries Used

Add inventory item and get single inventory item by ID

```
const item = new Inventory(itemData);
await item.save();
```

Get all inventory items with filters (category, status, search)

```
const items = await Inventory.find(filter).sort({ createdAt: -1 });
```

Get single inventory item by ID

```
const item = await Inventory.findOne({ _id: req.params.id, restaurantId });
```

Update inventory item

```
const updatedItem = await Inventory.findByIdAndUpdate(
  itemId,
  updateData,
  { new: true, runValidators: true }
);
```

Update stock (add or subtract quantity)

```
const updatedItem = await Inventory.findByIdAndUpdate(
  itemId,
  {
    currentQuantity: newQuantity,
    updatedAt: Date.now()
  },
  { new: true, runValidators: true }
);
```

Delete inventory item

```
await Inventory.findByIdAndDelete(itemId);
```

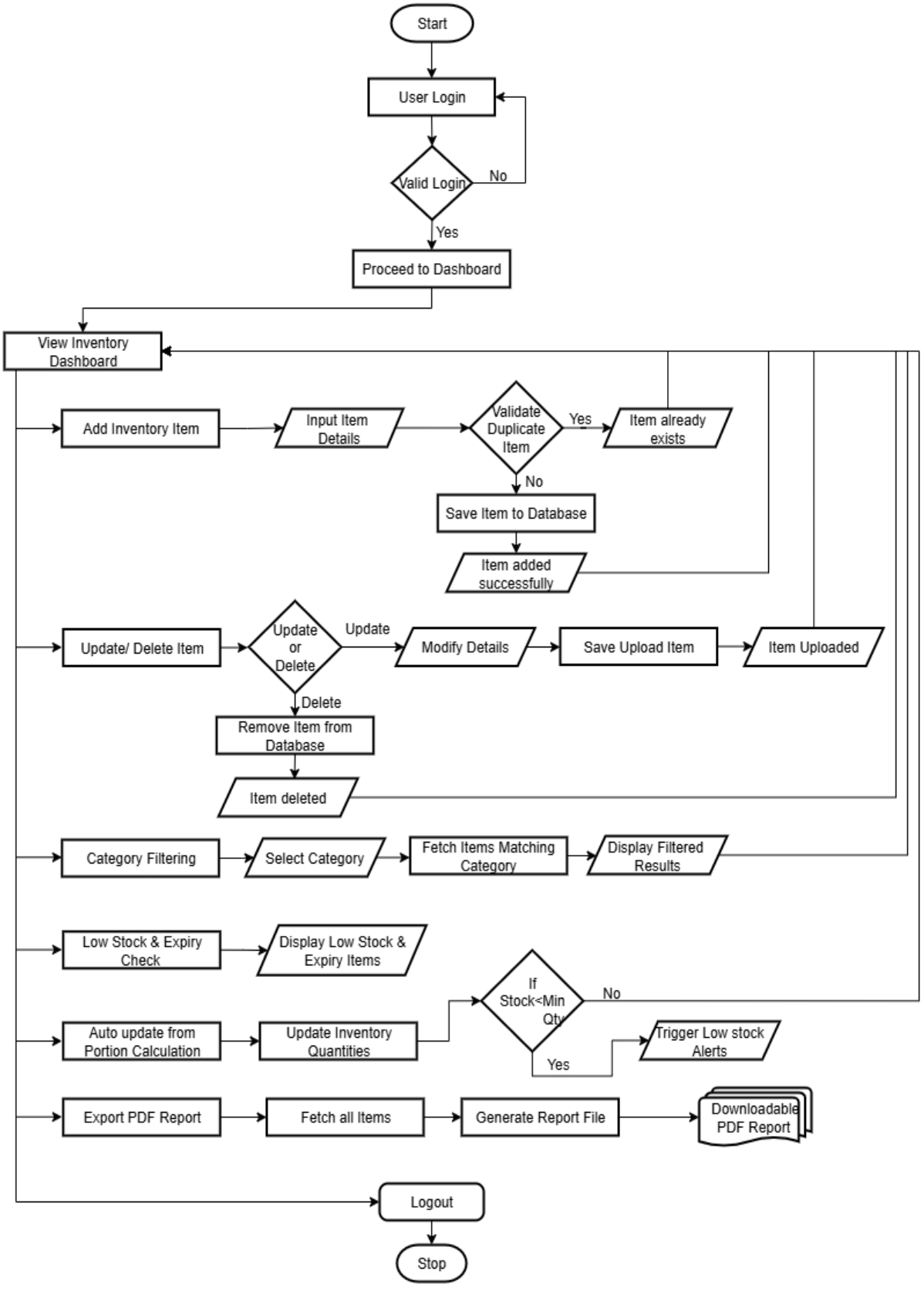
Get inventory statistics

```
const items = await Inventory.find({ restaurantId, isActive: true });
```

Get low stock alerts

```
const lowStockItems = await Inventory.find({
  restaurantId,
  isActive: true,
  $expr: { $lte: ['$currentQuantity', '$minQuantity'] }
}).sort({ currentQuantity: 1 });
```

Flow Chart:


Start
User Login
Valid Login
No
Yes
Proceed to Dashboard
View Inventory Dashboard
Add Inventory Item
Input Item Details
Validate Duplicate Item
Yes
Item already exists
No
Save Item to Database
Item added successfully
Update/ Delete Item
Update or Delete
Update
Modify Details
Save Upload Item
Item Uploaded
Delete
Remove Item from Database
Item deleted
Category Filtering
Select Category
Fetch Items Matching Category
Display Filtered Results
Low Stock & Expiry Check
Display Low Stock & Expiry Items
Auto update from Portion Calculation
Update Inventory Quantities
If Stock<Min Qty
No
Yes
Trigger Low stock Alerts
Export PDF Report
Fetch all Items
Generate Report File
Downloadable PDF Report
Logout
Stop

Portion Calculation Module (IT23858398 - Dilranga M.M.P.M)

Completion Level

The **Portion Calculation Module** is currently **80% completed**. Most of the essential functionalities have been implemented and tested, while a few improvements and refinements are still pending before the final version.

Completion Level	In progress
Portion calculation has been implemented, allowing meal plans to be adjusted based on the number of people.	Portion plans can now be saved for future reuse, improving efficiency and consistency
Ingredient quantities are now auto-calculated, ensuring accurate measurement for each portion.	
Inventory auto updates by deducting the exact amount of ingredients used during portion creation. Grocery and inventory list generation has been added, customized according to user type.	
Notification alerts are sent for inventory changes triggered by portion plans.	
Low stock alerts have been introduced to help users manage supplies proactively.	

Queries:

Create Portion Plan & Validate meal recipe

```
const createPortionPlan = async (req, res) => {
  // Validate main meal recipe
  const mainMealRecipe = await Recipe.findById(mainMeal.recipeId)
  .populate('ingredients.inventoryItemId');
  if (!mainMealRecipe) {
    return res.status(404).json({ message: 'Main meal recipe not found' });
  }
```

Validate Curry Recipe

```
const curryRecipes = await Recipe.find({
  _id: { $in: curries.map(curry => curry.recipeId) }
}).populate('ingredients.inventoryItemId');

if (curryRecipes.length !== curries.length) {
  return res.status(404).json({ message: 'One or more curry recipes not found' });
}
```

Create Portion Plan & Save Portion Plan

```
const portionPlan = new PortionPlan({
  restaurantId,
  name,
  mainMeal: {
    recipeId: mainMeal.recipeId,
    name: mainMealRecipe.name,
    servings: mainMealRecipe.servings
  },
  curries: curryRecipes.map((recipe, index) => ({
    recipeId: recipe._id,
    name: recipe.name,
    servings: recipe.servings
  })),
  peopleCount,
  totalIngredients,
  userType: userType || 'restaurant'
});
```

```
await portionPlan.save();
console.log('Portion plan saved with total cost:', portionPlan.totalCost);
```

Get All Portion Plans

```
const getAllPortionPlans = async (req, res) => {
  try {
    const restaurantId = req.user?.userId ? new mongoose.Types.ObjectId(req.user.userId) : null;

    let filter = {};
    if (restaurantId) {
      filter.restaurantId = restaurantId;
    }

    const plans = await PortionPlan.find(filter)
      .populate('mainMeal.recipeId', 'name category imageUrl')
      .populate('curries.recipeId', 'name category imageUrl')
      .sort({ createdAt: -1 });

    res.json({
      message: 'Portion plans retrieved successfully',
      plans
    });
  } catch (error) {
    console.error('Get portion plans error:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
};
```

Check Inventory

```
const executePortionPlan = async (req, res) => {
  try {
    const plan = await PortionPlan.findById(req.params.id);
    if (!plan) {
      return res.status(404).json({ message: 'Portion plan not found' });
    }

    if (plan.isInventoryDeducted) {
      return res.status(400).json({ message: 'Inventory already deducted for this plan' });
    }
```

Get All Portion Plans

```
const getAllPortionPlans = async (req, res) => {
  try {
    const restaurantId = req.user?.userId ? new mongoose.Types.ObjectId(req.user.userId) : null;

    let filter = {};
    if (restaurantId) {
      filter.restaurantId = restaurantId;
    }

    const plans = await PortionPlan.find(filter)
      .populate('mainMeal.recipeId', 'name category imageUrl')
      .populate('curries.recipeId', 'name category imageUrl')
      .sort({ createdAt: -1 });

    res.json({
      message: 'Portion plans retrieved successfully',
      plans
    });
  } catch (error) {
    console.error('Get portion plans error:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
};
```

Get Portion Plan by ID

```
const getPortionPlanById = async (req, res) => {
  try {
    const plan = await PortionPlan.findById(req.params.id)
      .populate('mainMeal.recipeId', 'name category imageUrl ingredients')
      .populate('curries.recipeId', 'name category imageUrl ingredients')
      .populate('totalIngredients.inventoryItemId', 'name category currentQuantity');

    if (!plan) {
      return res.status(404).json({ message: 'Portion plan not found' });
    }

    res.json({
      message: 'Portion plan retrieved successfully',
      plan
    });
  } catch (error) {
    console.error('Get portion plan error:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
};
```

Find Portion Plan

```
const inventoryChecks = await Promise.all(
  plan.totalIngredients.map(async (ingredient) => {
    // First try with restaurantId filter if available
    let inventoryQuery = { _id: ingredient.inventoryItemId };
    if (restaurantId) {
      inventoryQuery.restaurantId = restaurantId;
    }

    let inventoryItem = await Inventory.findOne(inventoryQuery);
```

Deduct Inventory

```
// All items are available, proceed with deduction
console.log('All items available, proceeding with deduction...');

const deductionResults = [];

for (const ingredient of plan.totalIngredients) {
  try {
    // Try to find inventory item with restaurantId first if available
    let inventoryQuery = {
      _id: ingredient.inventoryItemId,
      currentQuantity: { $gte: ingredient.totalQuantity }
    };

    if (restaurantId) {
      inventoryQuery.restaurantId = restaurantId;
    }

    let result = await Inventory.findOneAndUpdate(
      inventoryQuery,
      { $inc: { currentQuantity: -ingredient.totalQuantity } },
      { new: true }
    );

    // If not found with restaurantId and we have a restaurantId, try without it
    if (!result && restaurantId) {
      inventoryQuery = {
        _id: ingredient.inventoryItemId,
        currentQuantity: { $gte: ingredient.totalQuantity }
      };

      result = await Inventory.findOneAndUpdate(
        inventoryQuery,
        { $inc: { currentQuantity: -ingredient.totalQuantity } },
        { new: true }
      );
    }
```

```
    if (!result) {
      throw new Error(`Failed to deduct ${ingredient.itemName} from inventory - insufficient quantity or item not found`);
    }

    console.log(`Successfully deducted ${ingredient.totalQuantity} ${ingredient.unit} of
      ${ingredient.itemName}. After: ${result.currentQuantity}`);

    deductionResults.push({
      item: ingredient.itemName,
      deducted: ingredient.totalQuantity,
      unit: ingredient.unit,
      afterQuantity: result.currentQuantity
    });

  } catch (deductionError) {
    console.error(`Error deducting ${ingredient.itemName}:`, deductionError);
    throw deductionError;
  }
}
```

Update Portion Plan

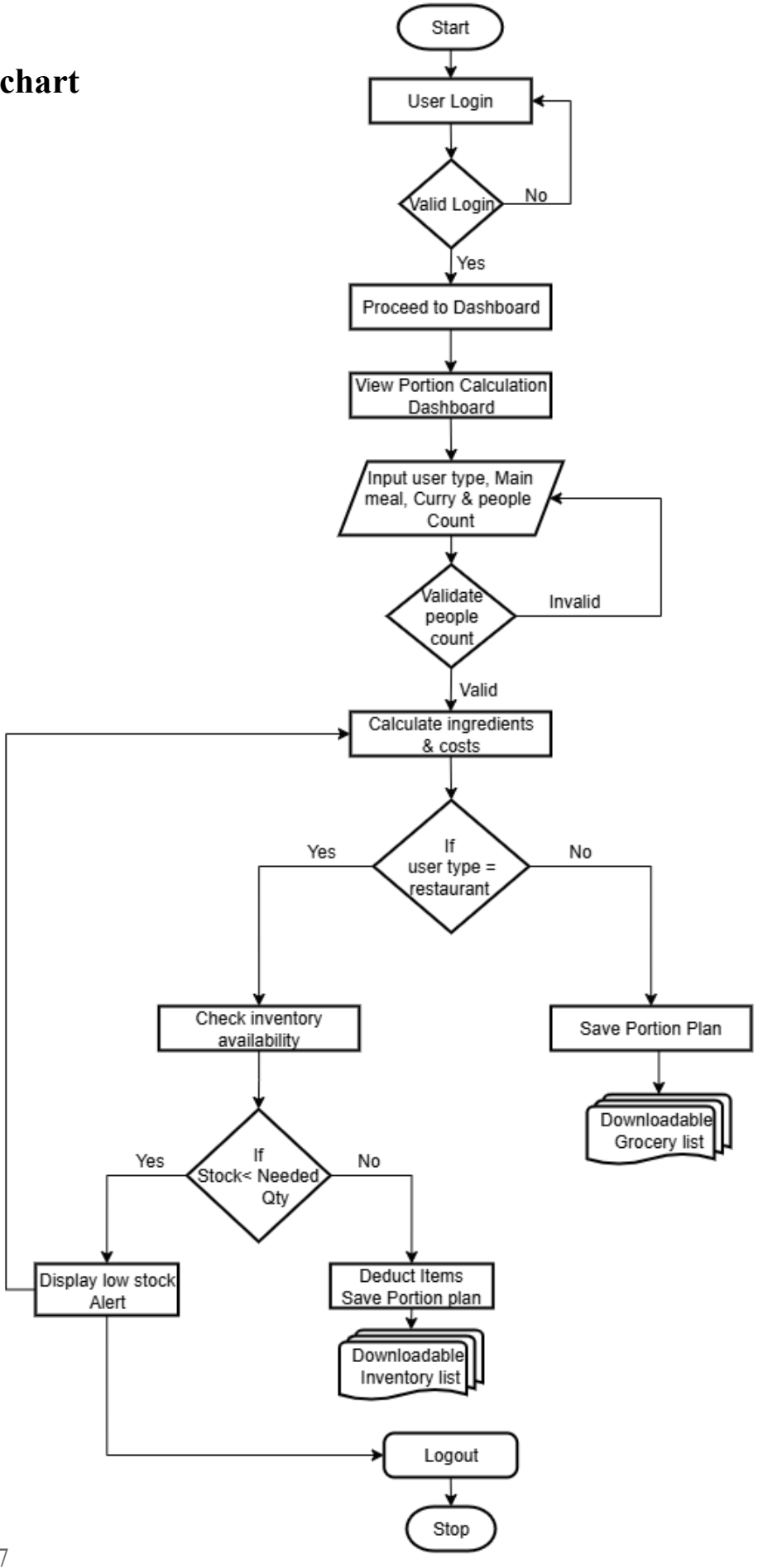
```
plan.isInventoryDeducted = true;
await plan.save();

console.log(`Successfully deducted inventory for plan ${plan.planId}:`, deductionResults);

return res.json({
  message: 'Portion plan executed and inventory deducted successfully',
  plan,
  deductionResults
});
```

Generate PDF & Delete Portion Plan

```
const generatePDF = async (req, res) => {
  try {
    const plan = await PortionPlan.findById(req.params.id)
      .populate('mainMeal.recipeId', 'name category')
      .populate('curries.recipeId', 'name category');

    if (!plan) {
      return res.status(404).json({ message: 'Portion plan not found' });
    }

    const now = new Date();
```

```
const deletePortionPlan = async (req, res) => {
  try {
    const plan = await PortionPlan.findByIdAndDelete(req.params.id);

    if (!plan) {
      return res.status(404).json({ message: 'Portion plan not found' });
    }

    res.json({
      message: 'Portion plan deleted successfully'
    });
  } catch (error) {
    console.error('Delete portion plan error:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
};
```

Flowchart


Start
User Login
Valid Login
No
Yes
Proceed to Dashboard
View Portion Calculation Dashboard
Input user type, Main meal, Curry & people Count
Validate people count
Invalid
Valid
Calculate ingredients & costs
If user type = restaurant
Yes
No
Check inventory availability
Save Portion Plan
Downloadable Grocery list
If Stock< Needed Qty
Yes
No
Display low stock Alert
Deduct Items Save Portion plan
Downloadable Inventory list
Logout
Stop

Recipe Management Module (IT23857230 - Damsara T.M.D.)

Completion Level

The Recipe Management Module is currently **80% completed**. Most of the CRUD operations, cost calculations, scaling and availability checks have been implemented and tested. Only minor refinements and UI/UX improvements are pending.

Completed	To be Completed
Add new recipes with cost calculation (per serving and total).	Send and receive notification from Inventory
Generate unique recipe ID automatically (`RCP0001`, `RCP0002`, …).	The **UI/UX design of the inventory dashboard** needs to be improved to enhance navigation, responsiveness and overall user friendliness
Export as PDF functionality has been implemented, making it easy to generate and download recipes reports.	
Update existing recipes with recalculated costs.	
Delete recipes securely (only owner can delete).	
Search, filter and retrieve all recipes with category, status and text search.	
Scale recipes by number of servings (dynamic recalculation).	

Queries Used:

Add inventory item & Get all recipes

```
const recipes = await Recipe.find(query).sort({ createdAt: -1 });
```

```
const addRecipe = async (req, res) => {
  try {
    const {
      name,
      description,
      category,
      subcategory,
      cuisine,
      servings,
      prepTime,
      cookTime,
      ingredients,
      imageUrl,
      status
    } = req.body;
```

Get single recipe item by ID

```
const recipe = await Recipe.findById(req.params.id);
```

Update recipes and Delete recipe

```
const updatedRecipe = await Recipe.findByIdAndUpdate(
  req.params.id,
  {
    name,
    description,
    category,
    subcategory,
    cuisine,
    servings,
    prepTime,
    cookTime,
    ingredients: processedIngredients,
    imageUrl,
    status,
    totalCost,
    costPerServing,
    updatedAt: new Date()
  },
  { new: true }
);
```

```
await Recipe.findByIdAndDelete(req.params.id);
```

Check Ingredient Availability

```
const { servings = 1 } = req.query;
const scalingFactor = servings / recipe.servings;

const availabilityCheck = [];

for (const ingredient of recipe.ingredients) {
  const inventoryItem = await Inventory.findById(ingredient.inventoryItemId);

  if (!inventoryItem) {
    availabilityCheck.push({
      itemName: ingredient.itemName,
      required: ingredient.quantity * scalingFactor,
      available: 0,
      unit: ingredient.unit,
      sufficient: false,
      status: 'not_found'
    });
    continue;
  }

  const requiredQuantity = ingredient.quantity * scalingFactor;
  const sufficient = inventoryItem.currentQuantity >= requiredQuantity;
```

Get inventory statistics

```
const getRecipeStats = async (req, res) => {
  try {
    const userId = req.user.id;

    const totalRecipes = await Recipe.countDocuments({ createdBy: userId });
    const curryRecipes = await Recipe.countDocuments({ createdBy: userId, category: 'curry' });
    const mealRecipes = await Recipe.countDocuments({ createdBy: userId, category: { $ne: 'curry' } });
```

Scale recipe

```
const recipe = await Recipe.findById(req.params.id);

if (!recipe) {
  return res.status(404).json({ message: 'Recipe not found' });
}

const { servings } = req.body;

if (!servings || servings <= 0) {
  return res.status(400).json({ message: 'Invalid servings amount' });
}

const scalingFactor = servings / recipe.servings;

const scaledIngredients = recipe.ingredients.map(ingredient => ({
  ...ingredient.toObject(),
  quantity: ingredient.quantity * scalingFactor
}));

const scaledRecipe = {
  ...recipe.toObject(),
  servings: servings,
  ingredients: scaledIngredients,
  totalCost: recipe.totalCost * scalingFactor,
  costPerServing: recipe.costPerServing
};
```

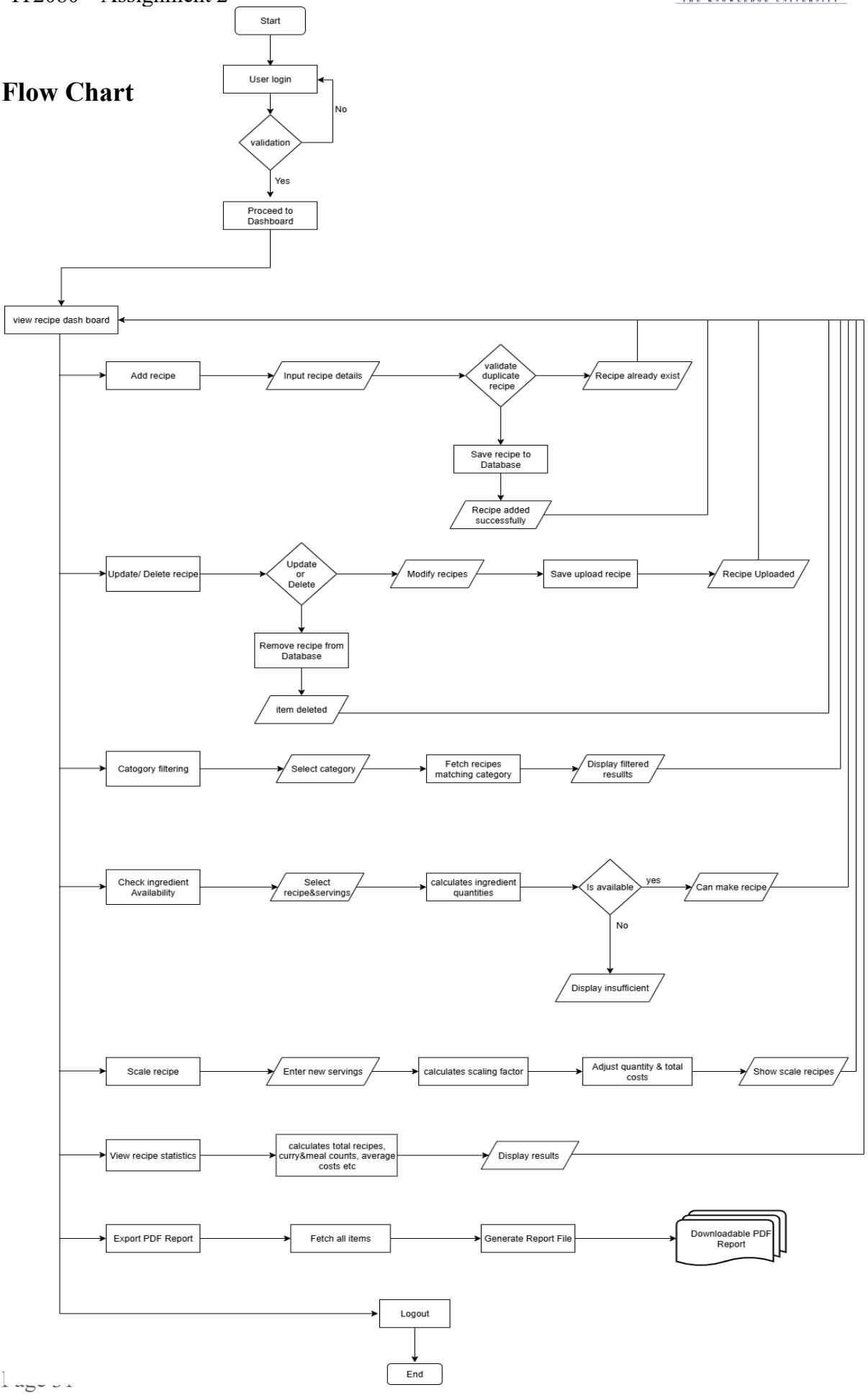
Get most used ingredients

```
const ingredientStats = await Recipe.aggregate([
  { $match: { createdBy: req.user._id } },
  { $unwind: '$ingredients' },
  { $group: {
      _id: '$ingredients.itemName',
      count: { $sum: 1 },
      totalQuantity: { $sum: '$ingredients.quantity' }
    }
  },
  { $sort: { count: -1 } },
  { $limit: 5 }
]);
```

Get recipes by category

```
const categoryStats = await Recipe.aggregate([
  { $match: { createdBy: req.user._id } },
  { $group: {
      _id: '$category',
      count: { $sum: 1 },
      avgCost: { $avg: '$costPerServing' }
    }
  },
  { $sort: { count: -1 } }
]);
```

Flow Chart


Start
User login
No
validation
Yes
Proceed to Dashboard
view recipe dash board
Add recipe
Input recipe details
validate duplicate recipe
Recipe already exist
Save recipe to Database
Recipe added successfully
Update/ Delete recipe
Update or Delete
Modify recipes
Save upload recipe
Recipe Uploaded
Remove recipe from Database
item deleted
Catogory filtering
Select category
Fetch recipes matching category
Display filtered resullts
Check ingredient Availability
Select recipe&servings
calculates ingredient quantities
Is available
yes
Can make recipe
No
Display insufficient
Scale recipe
Enter new servings
calculates scaling factor
Adjust quantity & total costs
Show scale recipes
View recipe statistics
calculates total recipes, curry&meal counts, average costs etc
Display results
Export PDF Report
Fetch all items
Generate Report File
Downloadable PDF Report
Logout
End

Leftover Management Module (IT23860032 - Ambagahawaththa N.S.)

Completion Level

The Leftover Management Module is currently **80% completed**. Most of the essential functionalities have been implemented and tested, while a few improvements and refinements are still pending before the final version.

Completed

- Add and manage leftover donations
- Export my donations and my requests as PDF reports.
- Get smart recipe suggestions and leftover usage tips via AI Chatbot.
- Filter donations by category and location to quickly find nearby requesters.

To be Completed

- Implement request donations to let users directly request needed leftovers.
- Add real time notifications to update users on admin approvals.
- Enhance the UI/UX to deliver a cleaner and user friendly experience.

Queries Used:

Get All Leftovers:

```
const leftovers = await Leftover.find(query)
  .populate('donorId', 'firstName lastName restaurantName email role')
  .populate('claimedBy.userId', 'firstName lastName restaurantName')
  .sort(sortObj)
  .skip(skip)
  .limit(parseInt(limit));

const total = await Leftover.countDocuments(query);
```

Get User's Leftovers:

```
const leftovers = await Leftover.find({ donorId: req.user.id })
  .populate('claimedBy.userId', 'firstName lastName restaurantName')
  .sort({ createdAt: -1 })
  .skip(skip)
  .limit(parseInt(limit));

const total = await Leftover.countDocuments({ donorId: req.user.id });
```

Create New Leftover & Get Leftover ID:

```
const leftover = new Leftover(leftoverData);
await leftover.save();
```

```
const leftover = await Leftover.findById(id);
```

Get Pending Leftovers & Create New Donation Request:

```
const leftovers = await Leftover.find(query)
  .populate('donorId', 'firstName lastName restaurantName email role')
  .sort({ createdAt: -1 })
  .skip(skip)
  .limit(parseInt(limit));

const total = await Leftover.countDocuments(query);
```

```
const donationRequest = new DonationRequest(requestData);
await donationRequest.save();
```

Get All Donation Request:

```
const requests = await DonationRequest.find(query)
  .populate('requesterId', 'firstName lastName email role')
  .sort({ urgencyLevel: -1, neededBy: 1 })
  .skip(skip)
  .limit(parseInt(limit));

const total = await DonationRequest.countDocuments(query);
```

Get All Donation Request & Get Pending Donation Requests (Admin):

```
const donationRequest = await DonationRequest.findById(requestId);
```

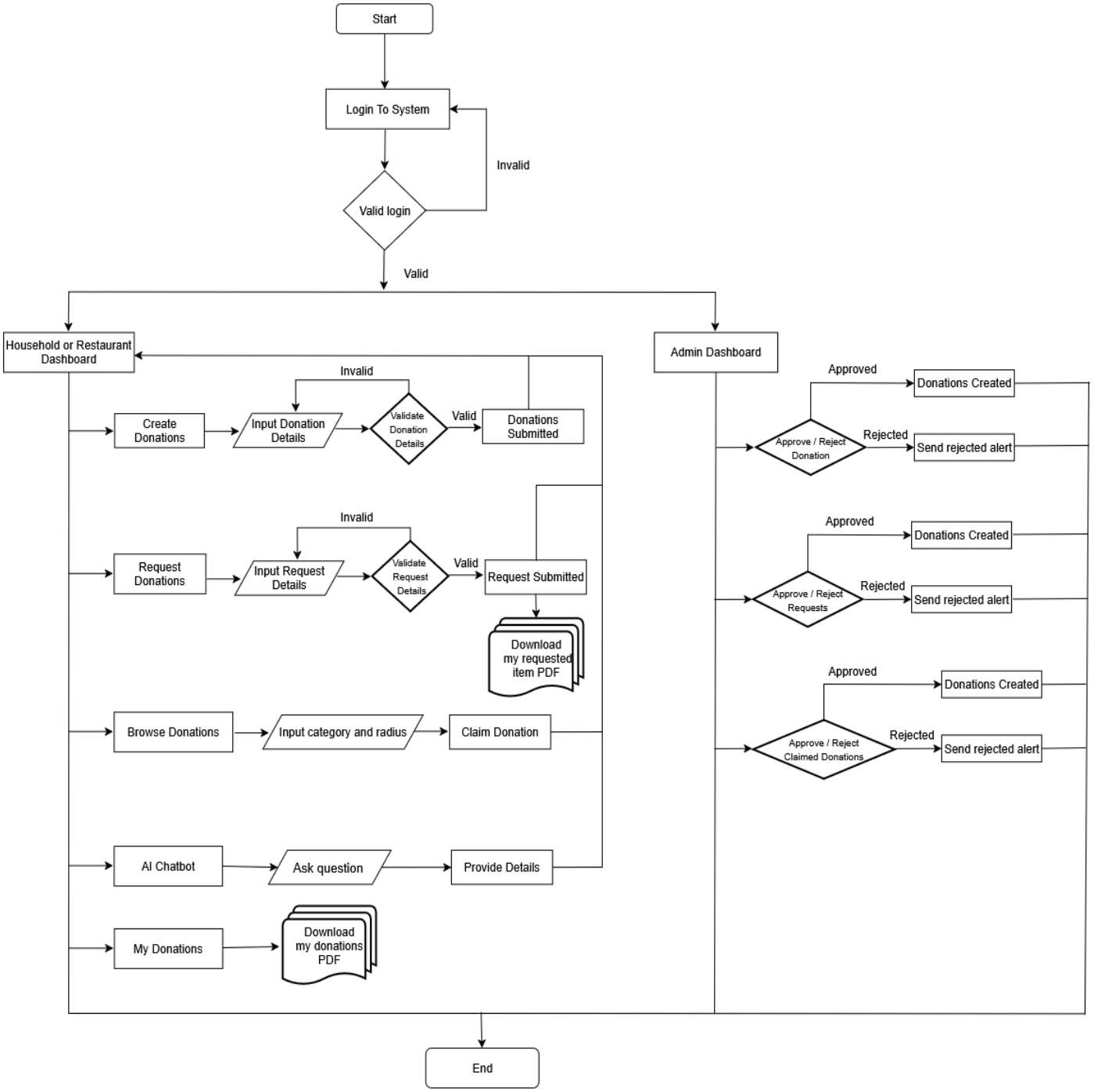
```
const requests = await DonationRequest.find({ status: 'pending' })
  .populate('requesterId', 'firstName lastName email role')
  .sort({ urgencyLevel: -1, createdAt: -1 })
  .skip(skip)
  .limit(parseInt(limit));

const total = await DonationRequest.countDocuments({ status: 'pending' });
```

Approve / Reject Donation Request:

```
const approveDonationRequest = async (req, res) => {
  try {
    // Only admins can approve/reject
    if (req.user.role !== 'admin' && req.user.userType !== 'admin') {
      return res.status(403).json({
        success: false,
        message: 'Access denied. Admin rights required.'
      });
    }

    const { id } = req.params;
    const { action, reason, adminNotes } = req.body;

    const donationRequest = await DonationRequest.findById(id);
    if (!donationRequest) {
      return res.status(404).json({
        success: false,
        message: 'Donation request not found'
      });
    }

    if (donationRequest.status !== 'pending') {
      return res.status(400).json({
        success: false,
        message: 'Donation request is not pending approval'
      });
    }
```

```
// Simple fix: always use a valid ObjectId
const adminId = new mongoose.Types.ObjectId(); // Creates a new valid ObjectId
const adminName = req.user.firstName && req.user.lastName
  ? `${req.user.firstName} ${req.user.lastName}`
  : req.user.name || req.user.username || req.user.email || 'Admin';

if (action === 'approve') {
  donationRequest.status = 'approved';
  donationRequest.approvedBy = {
    adminId,
    adminName,
    approvedAt: new Date(),
    adminNotes: adminNotes || ''
  };
} else if (action === 'reject') {
  donationRequest.status = 'rejected';
  donationRequest.rejectionReason = reason || 'Rejected by admin';
  donationRequest.approvedBy = {
    adminId,
    adminName,
    approvedAt: new Date(),
    adminNotes: adminNotes || ''
  };
} else {
  return res.status(400).json({
    success: false,
    message: 'Invalid action. Use "approve" or "reject"',
  });
}

await donationRequest.save();
```

Flow Chart:


Start
Login To System
Invalid
Valid login
Valid
Household or Restaurant Dashboard
Admin Dashboard
Invalid
Create Donations
Input Donation Details
Validate Donation Details
Valid
Donations Submitted
Approved
Donations Created
Approve / Reject Donation
Rejected
Send rejected alert
Invalid
Request Donations
Input Request Details
Validate Request Details
Valid
Request Submitted
Approved
Donations Created
Approve / Reject Requests
Rejected
Send rejected alert
Download my requested item PDF
Browse Donations
Input category and radius
Claim Donation
Approved
Donations Created
Approve / Reject Claimed Donations
Rejected
Send rejected alert
AI Chatbot
Ask question
Provide Details
My Donations
Download my donations PDF
End

Finance Management Module (IT23858398 - Dilranga M.M.P.M)

Completion Level

The **Finance Management Module** is currently **80% completed**. Most of the essential functionalities have been implemented and tested, while a few improvements and refinements are still pending before the final version.

Completion Level	In progress
Daily and monthly profit tracking has been implemented, showing today's profit, monthly profit, income, and expenses clearly.	Individual staff salary reports can now be downloaded for detailed record-keeping.
Staff management is enhanced with real-time staff count and total monthly salary budget display.	Portion creation is fully integrated with the finance system: each portion automatically appears in the records, adds profit to the financial summary, and is included in the overall finance record.
Current inventory value calculation has been integrated into the finance dashboard.	
Comprehensive staff payment management has been developed, covering salaries, allowances, deductions, bonuses, and automatic EPF/ETF calculations.	
PDF report generation is available, allowing users to export detailed finance records quickly.	
Finance records can now be viewed in detail with options to delete specific entries for better management.	

Queries Used:

Get daily profit

```
const records = await FinanceRecord.find({
  restaurantId: userId,
  date: { $gte: startOfDay, $lte: endOfDay }
}).populate('staffId', 'firstName lastName position');
```

Get monthly profit & Add finance record

```
// Get monthly data
const monthlyRecords = await FinanceRecord.find({
  restaurantId: userId,
  $expr: {
    $and: [
      { $eq: [{ $month: '$date' }, currentMonth] },
      { $eq: [{ $year: '$date' }, currentYear] }
    ]
  }
}).populate('staffId', 'firstName lastName photoUrl position');
```

```
const financeRecord = new FinanceRecord({
  restaurantId: userId,
  type,
  category,
  amount: parseFloat(amount),
  description,
  date: date ? new Date(date) : new Date(),
  paymentMethod: paymentMethod || 'cash',
  invoiceNumber,
  notes,
  staffId: staffId || undefined,
  createdBy: userId
});
```

Get all finance records (pagination + filters)

```
// Get all finance records with pagination
const getFinanceRecords = async (req, res) => {
  try {
    const { page = 1, limit = 20, type, category, startDate, endDate } = req.query;
    const userId = req.user?.id || req.user?._id;

    let filter = { restaurantId: userId };

    if (type) filter.type = type;
    if (category) filter.category = category;
    if (startDate && endDate) {
      filter.date = {
        $gte: new Date(startDate),
        $lte: new Date(endDate)
      };
    }

    const skip = (page - 1) * limit;

    const records = await FinanceRecord.find(filter)
      .populate('staffId', 'firstName lastName position photoUrl')
      .sort({ date: -1, createdAt: -1 })
      .skip(skip)
      .limit(parseInt(limit));
```

Get staff performance / attendance

```
// Get staff details
const staff = await Staff.findById(staffId);
if (!staff) {
  return res.status(404).json({
    success: false,
    message: 'Staff member not found'
  });
}

// Get attendance records for the specified month/year
const attendanceRecords = await Attendance.find({
  staffId: staffId,
  $expr: {
    $and: [
      { $eq: [{ $month: '$date' }, targetMonth] },
      { $eq: [{ $year: '$date' }, targetYear] }
    ]
  }
});
```

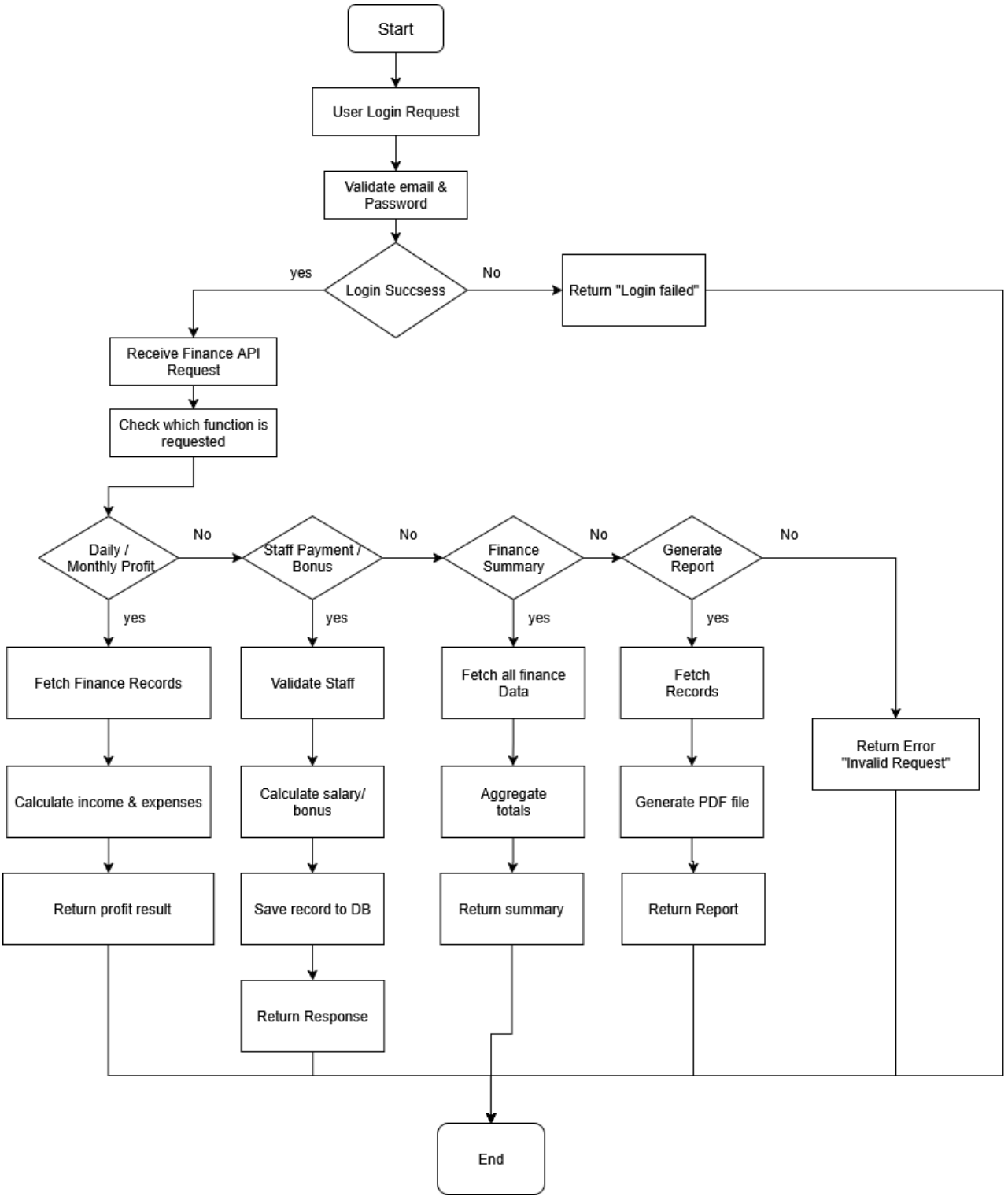
Get current inventory value

```
// Get inventory value
const inventoryValue = await Inventory.aggregate([
  { $match: { restaurantId: new mongoose.Types.ObjectId(userId) } },
  {
    $group: {
      _id: null,
      totalValue: { $sum: { $multiply: ['$currentQuantity', '$costPerUnit'] } }
    }
  }
]);
```

Staff Count / Salary Budget

```
// Get staff count and total salaries
const staffCount = await Staff.countDocuments({ restaurantId: userId });
const totalSalaryBudget = await Staff.aggregate([
  { $match: { restaurantId: new mongoose.Types.ObjectId(userId) } },
  { $group: { _id: null, totalSalary: { $sum: '$salary' } } }
]);
```

Flow Chart


Start
User Login Request
Validate email & Password
Login Succsess
yes
No
Return "Login failed"
Receive Finance API Request
Check which function is requested
Daily / Monthly Profit
No
Staff Payment / Bonus
No
Finance Summary
No
Generate Report
No
yes
yes
yes
yes
Fetch Finance Records
Validate Staff
Fetch all finance Data
Fetch Records
Return Error "Invalid Request"
Calculate income & expenses
Calculate salary/ bonus
Aggregate totals
Generate PDF file
Return profit result
Save record to DB
Return summary
Return Report
Return Response
End